Herlihy's Universal Construction In OCaml 5

Singaraju Venkat Koundinya
Siddharth George
Ventrapragada Krishna Sameer

Indian Institute of Technology Madras, Chennai

30. April 2026

Why this construction?

Motivation

- Designing thread-safe data structures is difficult.

Motivation

- Designing thread-safe data structures is difficult.
- Need correctness (Linearizability).

Motivation

- Designing thread-safe data structures is difficult.
- Need correctness (Linearizability).
- Need progress (lock-free/wait-free).

Motivation

- Designing thread-safe data structures is difficult.
- Need correctness (Linearizability).
- Need progress (lock-free/wait-free).
- There are custom designs for each data structure:
  - Treiber Stack
  - Michael-Scott Queue
  - Other Data structure-specific constructions

Motivation

- Designing thread-safe data structures is difficult.
- Need correctness (Linearizability).
- Need progress (lock-free/wait-free).
- There are custom designs for each data structure:
  - Treiber Stack
  - Michael-Scott Queue
  - Other Data structure-specific constructions

  Can we do better?

Herlihy's Universality Result

Theorem (Herlihy, 1991)

For any $n \geq 1$, any object with consensus number $\geq n$, together with atomic read/write registers, can be used to construct a wait-free linearizable implementation of any concurrent object for n threads.

Brief digression: What is Consensus?

- The Consensus problem:

Brief digression: What is Consensus?

- The Consensus problem:
  - Multiple threads propose values.

Brief digression: What is Consensus?

- The Consensus problem:
  - Multiple threads propose values.
  - They all agree on a single value.

Brief digression: What is Consensus?

- The Consensus problem:
  - Multiple threads propose values.
  - They all agree on a single value.
  - The value is among the proposed values.

Brief digression: What is Consensus?

- The Consensus problem:
  - Multiple threads propose values.
  - They all agree on a single value.
  - The value is among the proposed values.
  - These conditions are to be met even under races.

Brief digression: What is Consensus?

- The Consensus problem:
  - Multiple threads propose values.
  - They all agree on a single value.
  - The value is among the proposed values.
  - These conditions are to be met even under races.
- A synchronization object with a consensus number n, can solve the consensus problem for n threads in a wait-free manner.

Brief digression: What is Consensus?

- The Consensus problem:
  - Multiple threads propose values.
  - They all agree on a single value.
  - The value is among the proposed values.
  - These conditions are to be met even under races.
- A synchronization object with a consensus number n, can solve the consensus problem for n threads in a wait-free manner.

We'll see how this is relevant soon.

The Construction

Core Idea: Global log of function calls

- We use a linked-list to implement our log.

Core Idea: Global log of function calls

- We use a linked-list to implement our log.
- When a thread calls a function:

Core Idea: Global log of function calls

- We use a linked-list to implement our log.
- When a thread calls a function:

Core Idea: Global log of function calls

- We use a linked-list to implement our log.
- When a thread calls a function:
  - Atomically append it to this log.

Core Idea: Global log of function calls

- We use a linked-list to implement our log.
- When a thread calls a function:
  - Atomically append it to this log.
  - Replay operations from the start.

Core Idea: Global log of function calls

- We use a linked-list to implement our log.
- When a thread calls a function:
  - Atomically append it to this log.
  - Replay operations from the start.
  - Compute the result.
- Notice how the contention is now on the log and not the object.

Core Idea: Global log of function calls

- We use a linked-list to implement our log.
- When a thread calls a function:
  - Atomically append it to this log.
  - Replay operations from the start.
  - Compute the result.
- Notice how the contention is now on the log and not the object.

Isn't this just FCFS?

Core Idea: Global log of function calls

- We use a linked-list to implement our log.
- When a thread calls a function:
  - Atomically append it to this log.
  - Replay operations from the start.
  - Compute the result.
- Notice how the contention is now on the log and not the object.

Isn't this just FCFS? Yes.

Core Idea: Global log of function calls

- We use a linked-list to implement our log.
- When a thread calls a function:
  - Atomically append it to this log.
  - Replay operations from the start.
  - Compute the result.
- Notice how the contention is now on the log and not the object.

Isn't this just FCFS? Yes.
What about this is concurrent?

Core Idea: Global log of function calls

- We use a linked-list to implement our log.
- When a thread calls a function:
  - Atomically append it to this log.
  - Replay operations from the start.
  - Compute the result.
- Notice how the contention is now on the log and not the object.

Isn't this just FCFS? Yes.
What about this is concurrent? There's more to the construction.

Concurrency support: Log-head array

- Maintain an n-length array, where n is the number of threads.

Concurrency support: Log-head array

- Maintain an n-length array, where n is the number of threads.
- Each array slot points to a node in the global log.

Concurrency support: Log-head array

- Maintain an n-length array, where n is the number of threads.
- Each array slot points to a node in the global log.
- Each thread accesses its own slot (Can be parallelized!).

Concurrency support: Log-head array

- Maintain an n-length array, where n is the number of threads.
- Each array slot points to a node in the global log.
- Each thread accesses its own slot (Can be parallelized!).
- Whenever a thread wants to call a function:

Concurrency support: Log-head array

- Maintain an n-length array, where n is the number of threads.
- Each array slot points to a node in the global log.
- Each thread accesses its own slot (Can be parallelized!).
- Whenever a thread wants to call a function:
  - Create a node holding the function invocation.

Concurrency support: Log-head array

- Maintain an n-length array, where n is the number of threads.
- Each array slot points to a node in the global log.
- Each thread accesses its own slot (Can be parallelized!).
- Whenever a thread wants to call a function:
  - Create a node holding the function invocation.
  - Scan the array to find the most recent log-entry.

Concurrency support: Log-head array

- Maintain an n-length array, where n is the number of threads.
- Each array slot points to a node in the global log.
- Each thread accesses its own slot (Can be parallelized!).
- Whenever a thread wants to call a function:
  - Create a node holding the function invocation.
  - Scan the array to find the most recent log-entry.
  - Try appending its node to it.

Concurrency support: Log-head array

- Maintain an n-length array, where n is the number of threads.
- Each array slot points to a node in the global log.
- Each thread accesses its own slot (Can be parallelized!).
- Whenever a thread wants to call a function:
  - Create a node holding the function invocation.
  - Scan the array to find the most recent log-entry.
  - Try appending its node to it.
  - If it success, run all the functions on the base object till its own node.

Concurrency support: Log-head array

- Maintain an n-length array, where n is the number of threads.
- Each array slot points to a node in the global log.
- Each thread accesses its own slot (Can be parallelized!).
- Whenever a thread wants to call a function:
  - Create a node holding the function invocation.
  - Scan the array to find the most recent log-entry.
  - Try appending its node to it.
  - If it success, run all the functions on the base object till its own node.
  - Retry upon failure.

Concurrency support: Log-head array

- Maintain an n-length array, where n is the number of threads.
- Each array slot points to a node in the global log.
- Each thread accesses its own slot (Can be parallelized!).
- Whenever a thread wants to call a function:
  - Create a node holding the function invocation.
  - Scan the array to find the most recent log-entry.
  - Try appending its node to it.
  - If it success, run all the functions on the base object till its own node.
  - Retry upon failure.

Think where consensus come in.

OCaml 5 Implementation

Consensus Protocol

- It can be shown that CAS has consensus number ∞
- It can solve consensus for any number of processes
- The decide operation is:

```
let decide consensus value tid =
  propose consensus value tid;
  if Atomic.compare_and_set consensus.winner (-1) tid then begin
    match Atomic.get consensus.proposed.(tid) with
    | Some v -> v
    | None -> failwith "This should never happen"
  end
  else begin
    let winner_tid = Atomic.get consensus.winner in
    match Atomic.get consensus.proposed.(winner_tid) with
    | Some v -> v
    | None -> failwith "This should never happen"
  end
```

Log entries

- The log implemented as a linked list, and the following is the implementation and some important functions.

```
type ('a,'b) t =
{
  decide_next : ('a,'b) t CASConsensus.t;
  next : ('a,'b) t option Atomic.t;
  seq : int Atomic.t;
  invoc : ('a -> ('a * 'b)) option
}

let create invoc num_threads =
{
  decide_next = CASConsensus.create num_threads;
  next = Atomic.make None;
  seq = Atomic.make 0;
  invoc;
}
```

```
let max arr =
 let rec aux acc i =
  if i >= Array.length arr then acc
  else let cur = arr.(i) in
   if Atomic.get cur.seq > Atomic.get acc.seq
   then aux cur (i+1)
   else aux acc (i+1)
 in
 aux arr.(0) 1
```

Caveats

Owing to space constraints and for clarity, this logic is presented as pseudo-code. However, the following are expected for our implementation:

- The sequential object in its base form should be provided explicitly.
  Ex: An empty queue, root node of a bst etc.
- All the functions of the object need to be abstracted into a single `apply` function.
- For a function to be invoked, `apply` should be used with a parameter indicating the specific function to be called.

Lock-free Implementation

```
apply(lfu_obj, new_obj, invoc)

  tid ← getTID()
  prefer ← createNode(invoc)
  while prefer.seq = 0
    before ← max(head)
    after ←
before.decide_next.decide(prefer)
    before.next ← after
    after.seq ← before.seq + 1
    head[tid] ← after
  end while
current ← tail.next
acc ← new_obj
while current ≠ prefer
  acc.apply(current.invoc)
  current ← current.next
end while
return acc.apply(prefer.invoc)
```

Issues with lock-free implementation:

- Clearly, a thread can keep losing the consensus and can starve.

Issues with lock-free implementation:

- Clearly, a thread can keep losing the consensus and can starve.
- This implementation is just lock-free for a reason.

Issues with lock-free implementation:

- Clearly, a thread can keep losing the consensus and can starve.
- This implementation is just lock-free for a reason.
- We make a few modifications to the lock-free implementation to make it wait-free.

Issues with lock-free implementation:

- Clearly, a thread can keep losing the consensus and can starve.
- This implementation is just lock-free for a reason.
- We make a few modifications to the lock-free implementation to make it wait-free.
- On a high-level, the threads `help` each other out.

Conversion to wait-free implementation:

- Whenever a thread wants to append a node, it announces this to all other nodes.

Conversion to wait-free implementation:

- Whenever a thread wants to append a node, it announces this to all other nodes.
- This is done by maintaining a new array, called announce array.

Conversion to wait-free implementation:

- Whenever a thread wants to append a node, it announces this to all other nodes.
- This is done by maintaining a new array, called announce array.
- When it enters the first loop, it first checks if some other node wants to append itself.

Conversion to wait-free implementation:

- Whenever a thread wants to append a node, it announces this to all other nodes.
- This is done by maintaining a new array, called announce array.
- When it enters the first loop, it first checks if some other node wants to append itself.
- If there is one, this thread tries appending it, otherwise, it appends its own node.

Conversion to wait-free implementation:

- Whenever a thread wants to append a node, it announces this to all other nodes.
- This is done by maintaining a new array, called announce array.
- When it enters the first loop, it first checks if some other node wants to append itself.
- If there is one, this thread tries appending it, otherwise, it appends its own node.
- This ensures every thread's node gets appended in a finite number of steps.

Wait-free Implementation

apply$(wfu_obj,\ new_obj,\ invoc)$

```
tid ← getTID()
anc ← createNode(invoc)
announce[tid] ← anc
head[tid] ← max(head)
while announce[tid].seq = 0
   before ← head[tid]
   idx ← (before.seq + 1) mod n
   help ← announce[idx]
   if help.seq = 0
         prefer ← help
   else
         prefer ← announce[tid]
   after ← before.decide_next.decide(prefer)
   before.next ← after
   after.seq ← before.seq + 1
   head[tid] ← after
end while
```

```
head[tid] ← announce[tid]
current ← tail.next
acc ← new_obj
while current ≠ announce[tid]
   acc.apply(current.invoc)
   current ← current.next
end while
return acc.apply(current.invoc)
```

How is it wait-free?

- Inside the loop, each thread checks `announce[(before.seq + 1) % n]` - this round-robins over all thread-ids.

How is it wait-free?

- Inside the loop, each thread checks `announce[(before.seq + 1) % n]` - this round-robins over all thread-ids.
- If this node hasn't been appended yet, the threads attempt to append the it.

How is it wait-free?

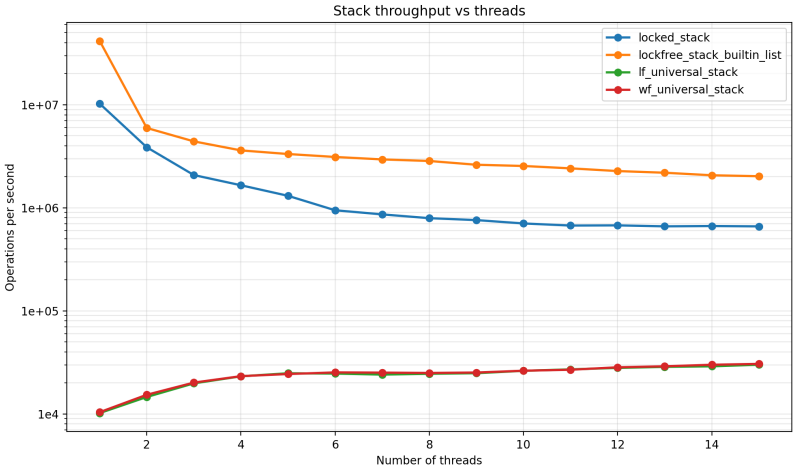
- Inside the loop, each thread checks `announce[(before.seq + 1) % n]` - this round-robins over all thread-ids.
- If this node hasn't been appended yet, the threads attempt to append the it.
- As length of the log keeps increasing, the node that gets helped keeps changing in a deterministic way.

How is it wait-free?

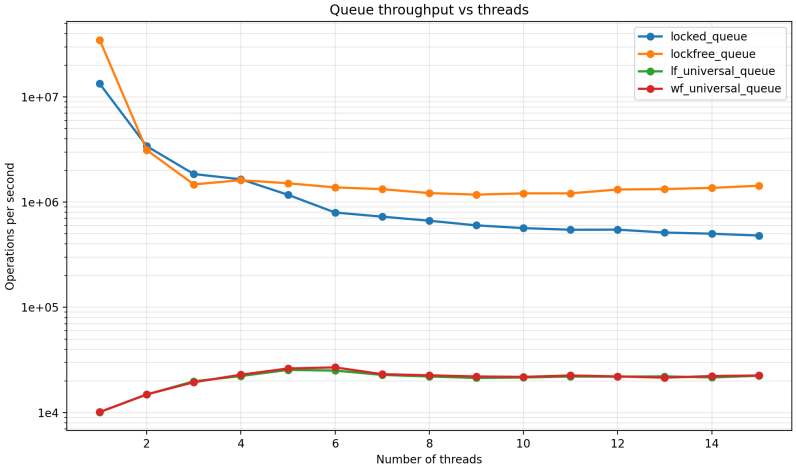
- Inside the loop, each thread checks `announce[(before.seq + 1) % n]` - this round-robins over all thread-ids.
- If this node hasn't been appended yet, the threads attempt to append the it.
- As length of the log keeps increasing, the node that gets helped keeps changing in a deterministic way.
- This ensures that any waiting node gets appended in at most n steps.

Benchmarking

Comparison with other given implementations



Comparison with other given implementations



Comparison with other implementations

- Let's see some possible questions:

Comparison with other implementations

- Let's see some possible questions:
- Why are the lock-free and wait-free implementations so slow?

Comparison with other implementations

- Let's see some possible questions:
- Why are the lock-free and wait-free implementations so slow?
  - Both the implementations are bottle-necked by the same thing - the $O(n^2)$ history replay. This has a significant overhead and consequently the throughput is low.

Comparison with other implementations

- Let's see some possible questions:
- Why are the lock-free and wait-free implementations so slow?
  - Both the implementations are bottle-necked by the same thing - the $O(n^2)$ history replay. This has a significant overhead and consequently the throughput is low.
- Why are they almost entirely overlapping?

Comparison with other implementations

- Let's see some possible questions:
- Why are the lock-free and wait-free implementations so slow?
  - Both the implementations are bottle-necked by the same thing - the $O(n^2)$ history replay. This has a significant overhead and consequently the throughput is low.
- Why are they almost entirely overlapping?
  - The reason is again the same. Whatever difference there is between the two implementations is completely dominated by the replay overhead.

Comparison with other implementations

- Let's see some possible questions:
- Why are the lock-free and wait-free implementations so slow?
  - Both the implementations are bottle-necked by the same thing - the $O(n^2)$ history replay. This has a significant overhead and consequently the throughput is low.
- Why are they almost entirely overlapping?
  - The reason is again the same. Whatever difference there is between the two implementations is completely dominated by the replay overhead.
- What even is the point of doing this then?

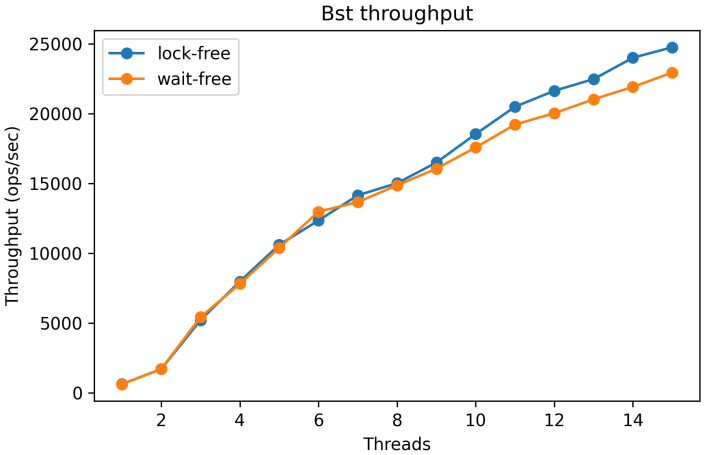
Comparison with other implementations

- Let's see some possible questions:
- Why are the lock-free and wait-free implementations so slow?
  - Both the implementations are bottle-necked by the same thing - the $O(n^2)$ history replay. This has a significant overhead and consequently the throughput is low.
- Why are they almost entirely overlapping?
  - The reason is again the same. Whatever difference there is between the two implementations is completely dominated by the replay overhead.
- What even is the point of doing this then?
  - Practically, there isn't any. But this construction shows that any sequential object can be converted into a wait-free linearizable concurrent object, albeit at the cost of speed.

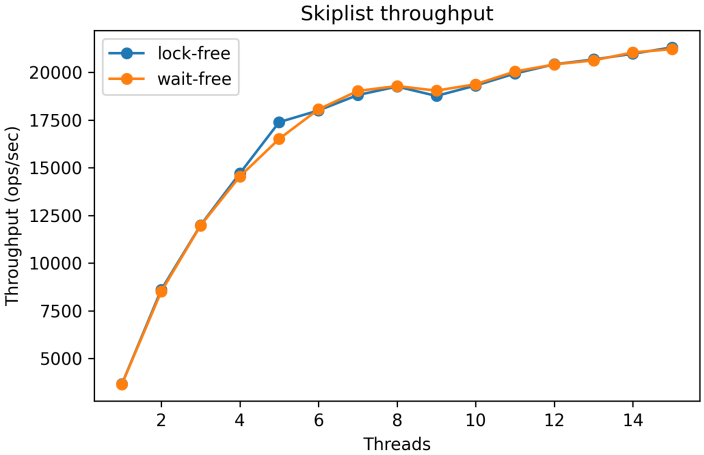
Comparison with other implementations

- Let's see some possible questions:
  - Why are the lock-free and wait-free implementations so slow?
    - Both the implementations are bottle-necked by the same thing - the $O(n^2)$ history replay. This has a significant overhead and consequently the throughput is low.
  - Why are they almost entirely overlapping?
    - The reason is again the same. Whatever difference there is between the two implementations is completely dominated by the replay overhead.
  - What even is the point of doing this then?
    - Practically, there isn't any. But this construction shows that any sequential object can be converted into a wait-free linearizable concurrent object, albeit at the cost of speed.
- Anyways, let's plot some more graphs, cuz why not?

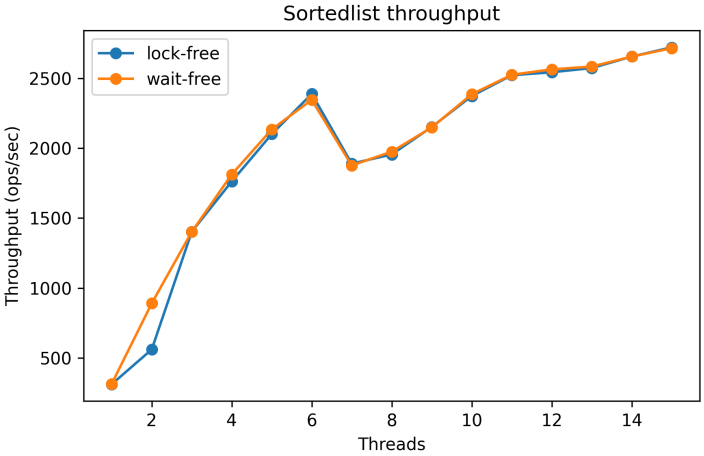
Binary Search Tree



Skiplist


Skiplist throughput
lock-free
wait-free
Throughput (ops/sec)
20000
17500
15000
12500
10000
7500
5000
2
4
6
8
10
12
14
Threads

Sortedlist


Sortedlist throughput
lock-free
wait-free
Throughput (ops/sec)
500
1000
1500
2000
2500
2
4
6
8
10
12
14
Threads

THANK YOU!